ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading in C++

[COURSE OUTCOME 3 (CO3)]
CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

ASSIGNMENT SPECIFICATIONS & STUDENT DETAILS

Field	Assignment Specification / Student Details
Assignment Title	Constructors, Destructors and Operator Overloading in C++
Total Marks	50 Marks
Nature of Assignment	Individual Laboratory Assignment
Duration	1 Week
Submission Date	07-11-2026
Bloom's Levels Assessed	K5 – Evaluate (Every task in this assignment is evaluated at K5 level)
Student Name	G. Vishnu Madhav
Register Number	192425192
Course Code & Title	DSA0117 - Object-Oriented Programming with C++
Roll / Batch	Batch 27
GitHub Repository Link	https://github.com/GVishnuMadhav1/DSA0117--C-G_Vishnu_Madhav_27/tree/main/Assignment%203%20CO%203

Mandatory Execution and GitHub Submission Guidelines

- Every student must individually execute every task in this assignment, including all 'Complete the Code' programs.
- Create a folder named Assignment 3 CO 3 in repository DSA0117--C-G_Vishnu_Madhav_27.
- Keep one folder for each task (Q1 to Q7), containing C++ source files, output text, and terminal screenshots.
- Include a comprehensive README.md with compiler version, compilation commands, and task result summaries.
- Commit your work progressively - at least one commit for each task.

MARK DISTRIBUTION TABLE

Task / Question	Bloom's Level	Max Marks
Q1 – Constructors and Constructor Overloading	K5 – Evaluate	6
Q2 – Copy Constructors: Shallow and Deep Copies	K5 – Evaluate	7
Q3 – Dynamic Constructors and Destructors	K5 – Evaluate	7
Q4 – Overloading Arithmetic and Stream Operators	K5 – Evaluate	7
Q5 – Overloading Increment, Comparison, Subscript	K5 – Evaluate	6
Q6 – The Rules of Operator Overloading	K5 – Evaluate	7

Q7 – Design a Complex-Number Class from First Principles	K5 – Evaluate	10
TOTAL EVALUATION	Bloom's Level K5	50

Q1 – CONSTRUCTORS AND CONSTRUCTOR OVERLOADING (6 MARKS)

1.1 Problem Context: Library Book Records

A library creates Book records in several situations: full details, donated books with only title, and empty records. To eliminate code duplication and prevent garbage values, constructor delegation is employed.

1.2 Member Initializer Lists vs Body Assignment

Member initializer lists initialize members directly upon construction rather than default-initializing and then assigning inside the body. This is mandatory for const members and reference members, and improves performance for class types by avoiding redundant default constructor calls. Member initialization follows declaration order, not initializer list order.

1.3 Explicit Constructors

Single-argument constructors act as implicit conversion operators unless marked explicit. Declaring explicit Book(int copies) prevents unintended conversions like Book b = 5;.

1.2 Complete C++ Source Code

```
#include <iostream>
#include <string>

class Book {
private:
    std::string title_;
    std::string author_;
    int copies_;

public:
    // 1. Default constructor delegating to parameterized
    Book() : Book("Untitled", "Anonymous", 0) {}

    // 2. Overloaded constructor for donated books
    Book(const std::string& title) : Book(title, "Donated", 1) {}

    // 3. Full parameterized constructor with member initializer list
    Book(const std::string& title, const std::string& author, int copies)
        : title_(title), author_(author), copies_(copies < 0 ? 0 : copies) {}

    // Explicit constructor to prevent implicit conversion
    explicit Book(int copies) : Book("Inventory Batch", "Internal", copies < 0 ? 0 : copies) {}

    void print() const {
        std::cout << "Book: '" << title_ << "' | Author: " << author_ << " | Copies: " << copies_ <<
'\n';
    }

    int getCopies() const { return copies_; }
};

int main() {
    std::cout << "============================================================\n";
    std::cout << " Q1: CONSTRUCTORS, OVERLOADING AND DELEGATION AUDIT\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    Book emptyRecord;
    Book donated("Clean Code");
    Book standardBook("Operating Systems", "Silberschatz", 15);
    Book negativeCopiesTest("Data Structures", "Weiss", -5); // Guarded
```

```
    Book explicitBatch(25); // Explicit conversion

    emptyRecord.print();
    donated.print();
    standardBook.print();
    negativeCopiesTest.print();
    explicitBatch.print();

    std::cout << "\n[VALIDATION] Negative copies safely clamped to 0.\n";
    std::cout << "[PASS] Constructor delegation and member initializer lists verified!\n";
    return 0;
}
```

[Screenshot: q01_screenshot.png]

Q2 – COPY CONSTRUCTORS: SHALLOW AND DEEP COPIES (7 MARKS)

2.1 Problem Context: Hospital Prescription Records

Prescription manages a dynamically allocated string array. When an object is copied without a custom copy constructor, the compiler performs a shallow copy: both objects point to the same heap address. When destructors execute, the second destructor attempts to delete already-freed memory, causing a fatal double-free crash.

2.2 The Rule of Three

If a class manages raw dynamically allocated memory, it must implement: 1. Destructor (to deallocate memory), 2. Copy Constructor (to allocate a new independent copy), and 3. Copy Assignment Operator (with self-assignment check and deallocation of old buffer).

2.3 Complete the Code: TODO 1 to 3 Verification

TODO 1 implemented deep copy constructor. TODO 2 implemented deep copy assignment operator with (this == &other) guard. TODO 3 implemented destructor delete[] medicines. Output matches expected: 'Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS'.

2.2 Complete C++ Source Code

```
#include <iostream>
#include <string>

class Prescription {
    int n;
    std::string* medicines; // dynamically allocated array

public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}

    // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every medicine (deep
copy)
    Prescription(const Prescription& other) : n(other.n), medicines(new std::string[other.n]) {
        for (int i = 0; i < n; ++i) {
            medicines[i] = other.medicines[i];
        }
    }

    // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy, release old
array and return *this
    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this; // Guard against self-assignment
        delete[] medicines; // Release existing memory
        n = other.n;
        medicines = new std::string[n];
        for (int i = 0; i < n; ++i) {
            medicines[i] = other.medicines[i];
        }
        return *this;
    }

    // TODO 3: Release the dynamically allocated array
    ~Prescription()
    {
        delete[] medicines;
    }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
```

```
int main()
{
    std::cout << "============================================================\n";
    std::cout << " Q2: DEEP COPY CONSTRUCTOR AND RULE OF THREE AUDIT\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p; // copy constructor (deep copy)
    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p; // copy assignment (deep copy)
    p.set(0, "Ibuprofen");

    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';

    std::cout << "Self-assignment test (p = p): ";
    p = p;
    std::cout << p.get(0) << ' ' << p.get(1) << " [SAFE]\n";
    std::cout << "[PASS] Expected output matched and independent deep copies verified!\n";
    return 0;
}
```

[Screenshot: q02_screenshot.png]

Q3 – DYNAMIC CONSTRUCTORS AND DESTRUCTORS (7 MARKS)

3.1 Dynamic Constructors and RAII in Auditorium Seat Booking

AuditoriumEvent allocates a 2D dynamic seat map using new char*[rows] and new char[seats]. Under the RAII (Resource Acquisition Is Initialization) idiom, memory allocation is tied to object lifetime; the destructor automatically frees all row buffers and pointer arrays upon exiting scope, preventing memory leaks.

3.2 Predict the Output Verification

[PREDICT THE OUTPUT VERIFICATION]

Predicted Output: +A +B -B copyAc use -Ac +P -P end -A

Actual Output: +A +B -B copyAc use -Ac +P -P end -A

Explanation: Block-scope object b is constructed (+B) and destroyed (-B). Passing a by value to use() invokes copy constructor (copyAc), and function parameter is destroyed upon return (-Ac). Dynamically allocated p is created (+P) and explicitly deleted (-P). Local object a is destroyed upon main exit (-A).

3.2 Complete C++ Source Code

```
#include <iostream>
#include <string>
#include <vector>

// --- PREDICT THE OUTPUT STRUCT ---
struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

// --- DYNAMIC AUDITORIUM SEAT MAP CLASS (RAII) ---
class AuditoriumEvent {
private:
    std::string eventName_;
    int rows_;
    int seatsPerRow_;
    char** seatMap_; // 2D dynamically allocated array

public:
    AuditoriumEvent(const std::string& name, int rows, int seats)
        : eventName_(name), rows_(rows), seatsPerRow_(seats) {
        if (rows_ <= 0 || seatsPerRow_ <= 0) {
            rows_ = 0; seatsPerRow_ = 0; seatMap_ = nullptr;
            return;
        }
        seatMap_ = new char*[rows_];
        for (int r = 0; r < rows_; ++r) {
            seatMap_[r] = new char[seatsPerRow_];
            for (int s = 0; s < seatsPerRow_; ++s) {
                seatMap_[r][s] = 'O'; // Available
            }
        }
    }

    ~AuditoriumEvent() {
        if (seatMap_) {
            for (int r = 0; r < rows_; ++r) {
```

```
                delete[] seatMap_[r];
            }
            delete[] seatMap_;
        }
    }

    bool bookSeat(int r, int s) {
        if (r < 0 || r >= rows_ || s < 0 || s >= seatsPerRow_) return false;
        if (seatMap_[r][s] == 'X') return false;
        seatMap_[r][s] = 'X';
        return true;
    }
};

int main()
{
    std::cout << "============================================================\n";
    std::cout << " Q3: DYNAMIC CONSTRUCTORS, DESTRUCTORS AND RAII AUDIT\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    std::cout << "--- 1. PREDICT THE OUTPUT EXECUTION ---\n";
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";

    std::cout << "\n\n--- 2. DYNAMIC 2D SEAT MAP ALLOCATION & BOOKING ---\n";
    {
        AuditoriumEvent symp("TechSymposium", 10, 20);
        symp.bookSeat(0, 0);
        symp.bookSeat(0, 1);
        std::cout << "Auditorium allocated: 10 rows x 20 seats (200 seats). Bookings active.\n";
        std::cout << "Exiting scope... RAII automatically releases all 2D dynamic buffers!\n";
    }
    std::cout << "[PASS] Memory management and destruction sequence validated!\n";
    return 0;
}
```

[Screenshot: q03_screenshot.png]

Q4 – OVERLOADING ARITHMETIC AND STREAM OPERATORS (7 MARKS)

4.1 Operator Overloading: Money Arithmetic

Money class stores currency as long long paise. Overloading operator+ allows intuitive bill calculation (c = a + b). operator+ returns a new Money value by copy, while operator+= returns a reference to *this.

4.2 Member vs Friend Operators

Binary operators where the left operand is a stream (e.g. std::cout << m) must be non-member functions because the left-hand operand is std::ostream, not Money. Declaring operator<< as a friend grants access to private member paise.

4.3 Complete the Code Resolution

TODO 1 (member operator+) and TODO 2 (member operator==, friend operator<<) verified. Output matches expected: 'Rs 200.00 \n true'. Carry test verified: Rs 0.75 + Rs 0.50 = Rs 1.25.

4.2 Complete C++ Source Code

```
#include <iomanip>
#include <iostream>

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: Overload + as a member function that returns a new Money object
    Money operator+(const Money& other) const {
        Money result;
        result.paise = this->paise + other.paise;
        return result;
    }

    Money operator-(const Money& other) const {
        Money result;
        result.paise = this->paise - other.paise;
        return result;
    }

    Money& operator+=(const Money& other) {
        this->paise += other.paise;
        return *this;
    }

    // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
    bool operator==(const Money& other) const {
        return this->paise == other.paise;
    }

    bool operator<(const Money& other) const {
        return this->paise < other.paise;
    }

    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        os << "Rs " << (m.paise / 100) << '.' << std::setw(2) << std::setfill('0') << (m.paise % 100) <<
std::setfill(' ');
        return os;
    }
};
```

```
int main()
{
    std::cout << "============================================================\n";
    std::cout << " Q4: OVERLOADING ARITHMETIC AND STREAM OPERATORS\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';

    std::cout << "--- CARRY TEST: Rs 0.75 + Rs 0.50 ---\n";
    Money p1(0, 75), p2(0, 50);
    Money pSum = p1 + p2;
    std::cout << p1 << " + " << p2 << " = " << pSum << " (Carried cleanly into Rs 1.25)\n";
    std::cout << "[PASS] Operator overloading +, ==, and << verified successfully!\n";
    return 0;
}
```

[Screenshot: q04_screenshot.png]

Q5 – OVERLOADING INCREMENT, COMPARISON AND SUBSCRIPT (6 MARKS)

5.1 Overloading Increment and Subscript Operators

[COUNTER PREDICTION VERIFICATION]
Predicted Output: 5 7 7
Actual Output: 5 7 7
Explanation: Postfix c++ returns old value 5 (assigned to a) and increments c to 6. Prefix ++c increments c to 7 and returns reference to c (7, assigned to b). Final c.v is 7.

Prefix vs Postfix: Postfix ++ takes a dummy int parameter: operator++(int). Postfix must create a temporary copy of the old state before incrementing, making it slower than prefix ++.

Subscript Operator []: Playlist provides both const and non-const operator[] with bounds checking, throwing std::out_of_range upon illegal index access.

5.2 Complete C++ Source Code

```
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>

// --- PREDICT THE OUTPUT COUNTER ---
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; }                    // prefix
    Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix
};

// Track Timer with Prefix and Postfix ++
class TrackTimer {
private:
    int hours_;
    int minutes_;
    int seconds_;

public:
    TrackTimer(int h = 0, int m = 0, int s = 0) : hours_(h), minutes_(m), seconds_(s) {}

    // Prefix ++: returns reference to updated object
    TrackTimer& operator++() {
        if (++seconds_ >= 60) {
            seconds_ = 0;
            if (++minutes_ >= 60) {
                minutes_ = 0;
                hours_++;
            }
        }
        return *this;
    }

    // Postfix ++: returns copy of old object
    TrackTimer operator++(int) {
        TrackTimer old = *this;
        ++(*this); // reuse prefix
        return old;
    }

    void print() const {
        std::cout << hours_ << ':' << minutes_ << ':' << seconds_ << '\n';
```

```
    }
};

// Playlist with Overloaded []
class Playlist {
private:
    std::vector<std::string> tracks_;

public:
    Playlist(std::initializer_list<std::string> list) : tracks_(list) {}

    // Non-const subscript
    std::string& operator[](int index) {
        if (index < 0 || index >= static_cast<int>(tracks_.size())) {
            throw std::out_of_range("Playlist index out of range!");
        }
        return tracks_[index];
    }

    // Const subscript
    const std::string& operator[](int index) const {
        if (index < 0 || index >= static_cast<int>(tracks_.size())) {
            throw std::out_of_range("Playlist index out of range!");
        }
        return tracks_[index];
    }

    int size() const { return static_cast<int>(tracks_.size()); }
};

int main()
{
    std::cout << "============================================================\n";
    std::cout << " Q5: OVERLOADING INCREMENT AND SUBSCRIPT OPERATORS\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    std::cout << "--- 1. PREDICT THE OUTPUT ---\n";
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << "\n\n";

    std::cout << "--- 2. ROLLOVER TEST: 00:59:59 -> 01:00:00 ---\n";
    TrackTimer timer(0, 59, 59);
    std::cout << "Before Increment : "; timer.print();
    ++timer;
    std::cout << "After Prefix ++  : "; timer.print();

    std::cout << "\n--- 3. PLAYLIST SUBSCRIPT [] WITH RANGE CHECK ---\n";
    Playlist pl = { "Track 1: Classical", "Track 2: Symphony", "Track 3: Rhapsody" };
    std::cout << "pl[0]: " << pl[0] << "\n";
    std::cout << "pl[2]: " << pl[2] << "\n";
    try {
        std::cout << "Accessing pl[-1]: ";
        std::cout << pl[-1] << "\n";
    } catch (const std::out_of_range& e) {
        std::cout << "[CAUGHT] " << e.what() << "\n";
    }
    std::cout << "[PASS] Increment, subscript and bounds safety verified!\n";
    return 0;
}
```

[Screenshot: q05_screenshot.png]

Q6 – THE RULES OF OPERATOR OVERLOADING (7 MARKS)

6.1 Rules of Operator Overloading

Rules Enforced by C++:

1. Non-overloadable operators: Scope resolution (::), Member access (.), Pointer to member (.*), Ternary (?:), sizeof, and typeid.
2. Arity and precedence cannot be altered: operator+ remains binary with level 4 precedence.
3. At least one operand must be a user-defined type.
4. Assignment (operator=), subscript ([]), call (()), and arrow (->) must be member functions.

Text Class Implementation: Custom text class manages dynamic char[] buffer for SMS gateway, overloading +, ==, and << with self-assignment protection.

6.2 Complete C++ Source Code

```
#include <iostream>
#include <cstring>

class Text {
private:
    char* data_;
    int len_;

public:
    Text() : data_(new char[1]{'\0'}), len_(0) {}
    Text(const char* s) : len_(std::strlen(s)), data_(new char[len_ + 1]) {
        std::strcpy(data_, s);
    }
    Text(const Text& other) : len_(other.len_), data_(new char[len_ + 1]) {
        std::strcpy(data_, other.data_);
    }
    Text& operator=(const Text& other) {
        if (this == &other) return *this;
        delete[] data_;
        len_ = other.len_;
        data_ = new char[len_ + 1];
        std::strcpy(data_, other.data_);
        return *this;
    }
    ~Text() { delete[] data_; }

    Text operator+(const Text& other) const {
        Text result;
        delete[] result.data_;
        result.len_ = this->len_ + other.len_;
        result.data_ = new char[result.len_ + 1];
        std::strcpy(result.data_, this->data_);
        std::strcat(result.data_, other.data_);
        return result;
    }

    bool operator==(const Text& other) const {
        return std::strcmp(this->data_, other.data_) == 0;
    }

    friend std::ostream& operator<<(std::ostream& os, const Text& t) {
        os << t.data_;
        return os;
    }
};
```

```
int main() {
    std::cout << "============================================================\n";
    std::cout << " Q6: RULES OF OPERATOR OVERLOADING & SMS GATEWAY TEXT\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    Text msg1("OTP: 4819. ");
    Text msg2("Valid for 10 minutes.");
    Text fullMsg = msg1 + msg2;
    std::cout << "Concatenated Message: " << fullMsg << "\n";

    Text copyMsg = fullMsg;
    std::cout << "Equality Check (copyMsg == fullMsg): " << std::boolalpha << (copyMsg == fullMsg) <<
"\n";
    copyMsg = copyMsg; // Self assignment safe
    std::cout << "After self assignment: " << copyMsg << "\n";
    std::cout << "[PASS] Rules of operator overloading verified!\n";
    return 0;
}
```

[Screenshot: q06_screenshot.png]

Q7 – DESIGN A COMPLEX-NUMBER CLASS FROM FIRST PRINCIPLES (10 MARKS)

7.1 AC Circuit Complex Impedance Calculator

Engineered Complex class from first principles representing AC impedances Z = R + jX. Implemented +, -, *, /, ==, << and magnitude().

Circuit Calculations:

- Series RLC: Z_series = ZR + ZL + ZC = 100 + 30j Ohm (Magnitude: 104.40 Ohm).
- Parallel RC: Z_parallel = (ZR * ZC) / (ZR + ZC) = 3.85 - 19.23j Ohm (Magnitude: 19.61 Ohm).
- Division by Zero: Division by zero impedance (0 + 0j) safely throws std::runtime_error.

7.2 Complete C++ Source Code

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <stdexcept>

class Complex {
private:
    double real_;
    double imag_;

public:
    Complex(double r = 0.0, double i = 0.0) : real_(r), imag_(i) {}

    double real() const { return real_; }
    double imag() const { return imag_; }
    double magnitude() const { return std::sqrt(real_ * real_ + imag_ * imag_); }

    Complex operator+(const Complex& o) const { return Complex(real_ + o.real_, imag_ + o.imag_); }
    Complex operator-(const Complex& o) const { return Complex(real_ - o.real_, imag_ - o.imag_); }
    Complex operator*(const Complex& o) const {
        return Complex(real_ * o.real_ - imag_ * o.imag_, real_ * o.imag_ + imag_ * o.real_);
    }
    Complex operator/(const Complex& o) const {
        double denom = o.real_ * o.real_ + o.imag_ * o.imag_;
        if (denom == 0.0) throw std::runtime_error("Division by zero impedance!");
        return Complex((real_ * o.real_ + imag_ * o.imag_) / denom,
                       (imag_ * o.real_ - real_ * o.imag_) / denom);
    }
    bool operator==(const Complex& o) const {
        return (std::abs(real_ - o.real_) < 1e-9) && (std::abs(imag_ - o.imag_) < 1e-9);
    }

    friend std::ostream& operator<<(std::ostream& os, const Complex& c) {
        os << std::fixed << std::setprecision(2) << c.real_
           << (c.imag_ >= 0 ? " + " : " - ") << std::abs(c.imag_) << "j";
        return os;
    }
};

int main() {
    std::cout << "============================================================\n";
    std::cout << " Q7: AC CIRCUIT IMPEDANCE CALCULATOR (COMPLEX CLASS)\n";
    std::cout << " Course: DSA0117 (CO3) | Student: G. Vishnu Madhav (192425192)\n";
    std::cout << "============================================================\n";

    // Circuit parameters: R = 100 Ohm, Inductive reactance XL = 50 Ohm, Capacitive reactance XC = 20
Ohm
    Complex zR(100.0, 0.0);
    Complex zL(0.0, 50.0);
```

```
    Complex zC(0.0, -20.0);

    // Series RLC Impedance: Z_series = ZR + ZL + ZC
    Complex zSeries = zR + zL + zC;
    std::cout << "Series RLC Impedance   : Z_series   = " << zSeries << " Ohm | Mag: " <<
zSeries.magnitude() << " Ohm\n";

    // Parallel RC Impedance: Z_parallel = (ZR * ZC) / (ZR + ZC)
    Complex zParallel = (zR * zC) / (zR + zC);
    std::cout << "Parallel RC Impedance  : Z_parallel = " << zParallel << " Ohm | Mag: " <<
zParallel.magnitude() << " Ohm\n";

    // Division by zero safety test
    try {
        Complex zZero(0.0, 0.0);
        Complex invalid = zR / zZero;
    } catch (const std::runtime_error& e) {
        std::cout << "Division by Zero Check : [TRAPPED] " << e.what() << "\n";
    }
    std::cout << "[PASS] AC Circuit Impedance Calculator completely validated!\n";
    return 0;
}
```

[Screenshot: q07_screenshot.png]